

AI-Assisted Black Box Testing 1

Dr Chris Meudec 2026-2038

A Basic Specification

"Consider a banking application allowing customers to transfer funds available from their account to another valid account. The application requires the user to enter the amount (whole euros only) to be transferred and the recipient's account number. Assume that the maximum amount that can be transferred per transaction is 10,000 € and that an account number is a 10-digit number. If invalid inputs are submitted, the transfer should be refused, and the error code 400 is returned. The application is to be tested at the API level. It returns 200 if the request has succeeded."

AI-Assisted Black Box Testing

[Using Microsoft CoPilot...](#)

[Using Claude Opus...](#)

Discussion

- the tests inputs, the expected behaviour
- the number of tests
- the justifications
- What does each say about an amount of exactly 10,000?
- About 0?
- About a transfer to your own account?
- About a well-formed account number that doesn't exist?
- About insufficient funds, which the spec mentions and then assigns no error code to?
- Which is better?
- Cyber attack injection in API (e.g. escape characters)
- **The big one: what's missing?**

Black Box Test Cases

- AKA functional or behavioural testing
- ideally **the full expected behaviour** of the system for each test is identified prior to the test execution (i.e. **test cases** rather than test inputs)
 - how often do we comply with this?
- test cases generation using Black box techniques is easier when the specification of the system or component **is clear and unambiguous** (at least rigorous, if not formal)
 - how often does this happen?
 - how does this work in agile?

Black Box Test Cases

- Test cases are entirely based on the software specification (which acts as a description of intended behaviour).
 - i.e. the code is not needed for the derivation of test cases (it is needed, of course, for test execution);
 - the program is considered a 'black-box'
 - For verification: design tests to have highest likelihood of failure
 - the generation of test cases can start as soon as the intended behaviour of a functionality is available (e.g. from use cases);
 - the test cases independent of the implementation language of the system.

Black Box Test Cases

- If created before designing or coding (e.g. using Test Driven Development), offers an opportunity to query the specification:
 - question our understanding of what needs to be implemented and improve it before wasting time trying to write the code.
- Test Cases can be generated from use cases or user stories as well as any form of description of behaviour.
 - The more detailed the specification is, the more thorough the black box test cases can be generated.
- Black box Tests are typically applied at a system level rather than at a unit level
 - Why?

The Oracle Problem



- **Where the difficulty actually lives: Three of these four boxes are automatable. One is not (or only very rarely).**



The Oracle Problem

DEFINITION

An oracle is any mechanism for deciding whether an observed output is correct.



Any mechanism

A document, a person, a second program, a mathematical property. Not necessarily code.



Not necessarily complete

An oracle may not check the complete behaviour.
That can be useful and dangerous at the same time.



Should always be present

Should be in test cases and assertions: encoding the oracle for automatic checking.

Kinds of oracle

Each one answers the question differently, and fails differently.

1 Specification

The requirement states what should happen. The expected output is read off the document.

Fails when: The spec is silent, ambiguous, or has drifted from what the business now wants.

2 Reference implementation

A previous version, a competitor's system, or a slow but obviously-correct model answers instead.

Fails when: Inherits every bug the reference has. "Different" is not the same as "wrong".

3 Human judgement

A domain expert looks at the output and decides. Still the oracle of last resort in most organisations.

Fails when: Does not scale, is not repeatable, and is not available to your CI pipeline at 3am.

4 Properties

Assert relationships the output must satisfy without knowing the exact value it should take.

Fails when: Partial by construction: only constrains the answer rather than use concrete values

AND ONE THAT IS NOT AN ORACLE AT ALL

“The code did this, so this must be right.”

This is not a weak oracle. It is a recording.

- ✗ It cannot disagree with the implementation, so it can never fail for a real reason.
- ✗ It carries no information about the requirement — none entered the process at any point.
- ✗ It promotes every bug currently in the code to the status of specified behaviour.
- ✗ It gets more expensive to remove the longer it stays, because now the bug is “expected”.

The tautological test

The failure mode that follows directly from using the code as its own oracle.

A test whose expected output was inferred from the implementation — guessed, read, or observed — rather than from any source of truth independent of it.

How one gets written

- 1 Spec says: amounts are whole euros only; invalid input returns 400.
- 2 Developer writes the parser and sends 100.00 to see what happens.
- 3 The API returns 200 — the parser silently accepted it.
- 4 Developer writes: `assertEquals(200, transfer(100.00).status);`

The result

- The test passes. It will always pass.
- Line coverage went up.
- The spec was violated and nothing fired.
- The defect is now protected: fixing the parser turns the suite red, so someone will “fix the test” instead.

Underdetermined specifications

The honest reason people go looking at the implementation.

A specification is underdetermined when it permits more than one answer.

From the transfer-API exercise

`amount = 100.00`

Whole euros only — but is a fractional literal with a whole value valid input or not? Both readings are defensible.

`Insufficient funds`

“Funds available” implies a balance check, but the spec never says what comes back. 400? 402? 422?

`Recipient = sender`

The spec is silent on self-transfer. Silence is not permission, and it is not prohibition either.

An underdetermined spec is a finding, not an obstacle. It is a defect in the requirement, discovered by test design, before a line of code is executed.

When the spec does not determine the answer

What to do instead, in this order.

1

Resolve it

Take it to whoever owns the requirement and get the document amended. The answer then becomes an oracle for everyone, not a private guess in one test file.

2

Assert only what every permitted answer shares

If 400, 402 and 422 are all legal, assert a 4xx and assert that no money moved. A weaker assertion that can fail beats a precise one that cannot.

3

Assert the property, not the value

Total balance across both accounts is unchanged. The same request twice does not move funds twice. No exact expected output required — this is Block 5.



Never: read the implementation and write down what it does. That is not a fourth option — it is the absence of one.

Exercise

- Use the same banking specification and produce not tests but a **specification defects list**:
 - ambiguities, omissions, contradictions: explain each problem individually and propose a resolution
 - Use AI but also your brain
 - Create Google Sheet and put it in [the shared class folder](#)

For now

- AI is useful for generating Test Cases if it has access to a solid specification. But its work must always be reviewed.
 - AI has no way to fully know if the specification is incomplete.
 - It may resolve ambiguity silently rather than reporting it.
 - Domain knowledge that isn't in the specification may not appear in the test cases it generates.
- No oracle means no test cases and no means of automatically verifying behaviour
- Remember, your job includes:
 - identifying the implicit requirements (e.g. in banking example: security, transfer of money must occur on success and not in invalid request)
 - and understanding the users' needs beyond what a specification says...